Laboratorium 3

Podstawy React + TypeScript + Vite

Aplikacje webowe i sieciowe

Cel laboratorium

Celem laboratorium jest przygotowanie aplikacji klienckiej `Cars Client` w technologii React + TypeScript. Aplikacja nie będzie jeszcze łączona z backendem ASP.NET Core. Dane będą udostępniane przez lokalny serwer `json-server`, który pozwoli przećwiczyć komponenty, props, state, formularze, komunikację HTTP i React Router.

Najważniejsze założenie

Na tym laboratorium skupiamy się na mechanizmach Reacta. Nie używamy jeszcze Axiosa ani globalnego magazynu stanu. Do komunikacji HTTP wystarczy wbudowana funkcja `fetch`.

Krok 1. Sprawdzenie środowiska

Upewnij się, że masz zainstalowane Node.js, npm oraz Visual Studio Code. Sprawdź wersje:

```
node --version
npm --version
```

Uwaga

Jeżeli Vite zgłosi problem z wersją Node.js, zaktualizuj Node.js do aktualnej wersji LTS i ponownie uruchom terminal.

Krok 2. Utworzenie projektu React + TypeScript + Vite

Utwórz projekt:

```
npm create vite@latest cars-client -- --template react-ts
cd cars-client
npm install
npm run dev
```

Vite wyświetli adres aplikacji, najczęściej:

`http://localhost:5173`

Otwórz projekt w VS Code:

```
code .
```

Krok 3. Uporządkowanie projektu

Usuń przykładowe elementy z szablonu Vite i przygotuj strukturę:

```
src/
  components/
  models/
  pages/
  services/
  layouts/
  App.tsx
  main.tsx
  index.css
```

- `components/` – mniejsze, wielokrotnie używane komponenty,
- `pages/` – widoki aplikacji,
- `models/` – typy i interfejsy TypeScript,
- `services/` – komunikacja z usługą REST,
- `layouts/` – wspólny układ aplikacji.

Krok 4. Instalacja React Router

Zainstaluj bibliotekę:

```
npm install react-router
```

W projekcie wykorzystamy m.in.: `BrowserRouter`, `Routes`, `Route`, `NavLink`, `Link`, `Outlet` i `useParams`.

Krok 5. Lokalny REST API z json-server

Zainstaluj pakiet:

```
npm install --save-dev json-server
```

W katalogu głównym projektu utwórz plik `db.json`:

```
{
  "cars": [
    {
      "id": "1",
      "brand": "Toyota",
      "model": "Corolla",
      "doorsNumber": 5,
      "luggageCapacity": 470,
      "engineCapacity": 1800,
      "fuelType": 1,
      "productionDate": "2022-05-10",
      "averageFuelConsumption": 6.4,
      "bodyType": 1
    },
    {
      "id": "2",
      "brand": "Skoda",
      "model": "Octavia",
```

```
      "doorsNumber": 5,
      "luggageCapacity": 600,
      "engineCapacity": 1500,
      "fuelType": 1,
      "productionDate": "2021-03-15",
      "averageFuelConsumption": 6.8,
      "bodyType": 1
    }
  ]
}
```

W `package.json` dopisz skrypt:

```
"api": "json-server db.json"
```

Przykładowa sekcja `scripts`:

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "api": "json-server db.json"
}
```

Uruchom serwer danych w osobnym terminalu:

```
npm run api
```

Sprawdź:

`http://localhost:3000/cars`

Efekt po kroku

`json-server` udostępnia lokalne REST API dla zasobu `cars`.

Krok 6. Model danych Car

W `src/models/Car.ts` przygotuj:

```
export interface Car {
  id: string;
  brand: string;
  model: string;
  doorsNumber: number;
  luggageCapacity: number;
  engineCapacity: number;
  fuelType: number;
  productionDate: string;
  averageFuelConsumption: number;
  bodyType: number;
}

export type NewCar = Omit<Car, 'id'>;
```

Uwaga

W aktualnym `json-server` identyfikator zasobu jest tekstem, dlatego `id` ma typ `string`. Datę przechowujemy jako `string`, ponieważ JSON nie ma osobnego typu `Date`.

Krok 7. Serwis do komunikacji z API

W `src/services/carService.ts` umieść logikę komunikacji HTTP:

```
import type { Car, NewCar } from '../models/Car';

const carsUrl = 'http://localhost:3000/cars';

async function ensureSuccess(response: Response) {
  if (!response.ok) {
    throw new Error(
      `Request failed. Status: ${response.status}`
    );
  }
}

export async function getCars(
  signal?: AbortSignal
): Promise<Car[]> {
  const response = await fetch(carsUrl, { signal });
  await ensureSuccess(response);
  return await response.json();
}

export async function getCarById(
  id: string,
  signal?: AbortSignal
): Promise<Car> {
  const response = await fetch(`${carsUrl}/${id}`, { signal });
  await ensureSuccess(response);
  return await response.json();
}

export async function createCar(
  car: NewCar
): Promise<Car> {
  const response = await fetch(carsUrl, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(car)
  });

  await ensureSuccess(response);
  return await response.json();
}

export async function updateCar(
  id: string,
  car: NewCar
): Promise<Car> {
  const response = await fetch(`${carsUrl}/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify(car)
  });

  await ensureSuccess(response);
  return await response.json();
}

export async function deleteCar(
  id: string
): Promise<void> {
  const response = await fetch(
    `${carsUrl}/${id}`,
    { method: 'DELETE' }
  );

  await ensureSuccess(response);
}
```

Wskazówka

Komponenty powinny odpowiadać głównie za interfejs i stan widoku. Szczegóły komunikacji HTTP trzymamy w osobnym serwisie.

Krok 8. Przygotowanie widoków

W folderze `pages` utwórz:

- `HomePage.tsx`,
- `CarsPage.tsx`,
- `CarDetailsPage.tsx`,
- `CreateCarPage.tsx`,
- `EditCarPage.tsx`,
- `NotFoundPage.tsx`.

W folderze `layouts` utwórz `RootLayout.tsx`. Powinien zawierać wspólne menu oraz `Outlet`.

Krok 9. Konfiguracja routingu

W `main.tsx`:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

W `App.tsx`:

```tsx
import { Route, Routes } from 'react-router';
import RootLayout from './layouts/RootLayout';
import HomePage from './pages/HomePage';
import CarsPage from './pages/CarsPage';
import CarDetailsPage from './pages/CarDetailsPage';
import CreateCarPage from './pages/CreateCarPage';
import EditCarPage from './pages/EditCarPage';
import NotFoundPage from './pages/NotFoundPage';

export default function App() {
  return (
    <Routes>
      <Route element={<RootLayout />}>
        <Route index element={<HomePage />} />

        <Route path="cars">
          <Route index element={<CarsPage />} />
          <Route path="new" element={<CreateCarPage />} />
          <Route path=":id" element={<CarDetailsPage />} />
          <Route path=":id/edit" element={<EditCarPage />} />
        </Route>

        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  );
}
```

Krok 10. Widok listy samochodów

Zaimplementuj `CarsPage`. Po wejściu na stronę:

1. pobierz listę przez `getCars()`,
2. przechowuj dane w `useState`,
3. użyj `useEffect` do pobrania danych po pojawieniu się komponentu,
4. obsłuż ładowanie, błąd i pustą listę.

Dla każdego samochodu wyświetl co najmniej markę i model oraz:

- **Szczegóły**,
- **Usuń**.

Krok 11. Usuwanie samochodu

Po kliknięciu *Usuń*:

1. zapytaj użytkownika o potwierdzenie,
2. po anulowaniu nie wykonuj żądania,
3. po potwierdzeniu wywołaj `deleteCar(id)`,
4. po sukcesie zaktualizuj listę.

Możesz użyć:

```
const confirmed = window.confirm(
  'Czy na pewno chcesz usun ten samochd?'
);
```

Krok 12. Widok szczegółów

W `CarDetailsPage`:

1. pobierz `id` przez `useParams()`,
2. pobierz samochód przez `getCarById(id)`,
3. wyświetl jego właściwości,
4. obsłuż ładowanie i błąd,
5. dodaj link powrotu do listy.

Krok 13. Formularz dodawania

W `CreateCarPage` przygotuj formularz kontrolowany. Dane pól przechowuj w stanie komponentu. Formularz powinien umożliwiać podanie:

- marki,
- modelu,
- liczby drzwi,
- pojemności bagażnika,
- pojemności silnika,
- rodzaju paliwa,
- daty produkcji,
- średniego spalania,
- typu nadwozia.

Po wysłaniu formularza:

1. użyj `preventDefault()`,
2. zbuduj obiekt `NewCar`,
3. wywołaj `createCar()`,
4. po sukcesie przejdź do listy lub widoku nowego samochodu.

Krok 14. Edycja samochodu

Utwórz widok `EditCarPage`, dostępny pod adresem:

`/cars/:id/edit`

Widok powinien umożliwiać zmianę danych istniejącego samochodu. Po wejściu na stronę:

1. pobierz `id` samochodu za pomocą `useParams()`,
2. pobierz jego aktualne dane przez `getCarById(id)`,
3. wypełnij nimi formularz kontrolowany,
4. po zatwierdzeniu zbuduj obiekt `NewCar`,
5. wywołaj `updateCar(id, car)`, które wysyła żądanie `PUT`,
6. po poprawnym zapisie przejdź do widoku szczegółów zaktualizowanego samochodu.

Dodaj również przycisk lub link **Edytuj**, prowadzący do `/cars/:id/edit`. Może znajdować się na liście samochodów, w widoku szczegółów albo w obu miejscach.

Uwaga

W tym laboratorium używamy `PUT`, czyli wysyłamy do serwera pełny nowy stan edytowanego samochodu.

Krok 15. Nawigacja

Menu zbuduj za pomocą `NavLink`. Powinno zawierać:

- Strona główna,
- Samochody,
- Dodaj samochód.

Krok 16. HomePage i NotFoundPage

`HomePage` może zawierać tytuł i krótki opis aplikacji.

`NotFoundPage` powinna informować o nieznanym adresie i zawierać link do strony głównej.

Krok 17. Proste stylowanie

Dodaj podstawowe CSS. Zadbaj o czytelność menu, listy, przycisków i formularza. Stylowanie nie jest głównym celem laboratorium.

Krok 18. Test aplikacji

Przetestuj:

1. stronę główną,
2. listę samochodów,
3. szczegóły,
4. anulowanie usuwania,
5. potwierdzenie usuwania,
6. dodawanie nowego samochodu,

7. edycję istniejącego samochodu,
8. stronę 404.

Podczas testowania obserwuj `Console`, `Network` oraz React Developer Tools.

Efekt końcowy

Po laboratorium masz aplikację React z routingiem, listą samochodów, szczegółami, formularzem dodawania i edycji, usuwaniem oraz lokalnym REST API.

Co dalej?

Na następnym laboratorium zastąpimy `json-server` prawdziwym backendem `Cars.API`.